Rapport de TP1 — API REST et Postman

Étudiant : Monsif Iguider
Date : 9 novembre 2026
API utilisée : DummyJSON (https://dummyjson.com)
Collection : TP1-API-REST-Postman.postman_collection.json

1. Objectif

Ce laboratoire met en pratique les échanges HTTP d'une API REST à l'aide de Postman : lecture, création, modification et suppression de ressources, utilisation des paramètres et des en-têtes, envoi de JSON, variables d'environnement et assertions automatisées.

2. Notions appliquées

Une requête HTTP comprend une méthode, une URL, éventuellement des en-têtes et un corps. La réponse comprend un code de statut et, généralement, des données JSON. REST identifie les ressources par leur URI et utilise les méthodes HTTP pour exprimer l'action. Une API stateless traite chaque requête à partir des informations fournies dans cette requête.

Méthode | Rôle observé dans le TP
GET | Lire une collection, un produit ou une recherche
POST | Envoyer les données d'une nouvelle ressource
PUT | Envoyer la représentation complète d'une ressource
PATCH | Envoyer seulement les champs à modifier
DELETE | Demander la suppression d'une ressource

3. Organisation Postman

La collection regroupe les requêtes dans les dossiers Lecture (GET), Création (POST), Modification (PUT/PATCH), Suppression (DELETE) et Erreurs et validation. L'environnement Demo API Env définit :

- baseUrl : https://dummyjson.com
- productId : 1
- createdProductId : renseigné par le script après la création simulée

Les requêtes utilisent Content-Type: application/json pour les corps JSON et Accept: application/json pour indiquer le format de réponse attendu.

4. Résultats et observations

Les endpoints ont été interrogés le 9 novembre 2026. Les statuts ci-dessous sont ceux observés lors de cette vérification ; une API publique peut évoluer et les résultats peuvent varier.

Opération | Requête | Statut observé | Observation
Lister les produits | `GET /products?limit=5` | 200 | Réponse JSON avec la liste `products` et des métadonnées de pagination
Lire un produit | `GET /products/1` | 200 | Réponse contenant l'identifiant et les propriétés du produit
Rechercher | `GET /products/search?q=phone` | 200 | Réponse structurée autour de `products`
Catégorie | `GET /products/category/smartphones` | 200 | La route de catégorie renvoie une liste
Créer un produit | `POST /products/add` | 201 observé | L'API renvoie un identifiant et les propriétés soumises
Modifier un produit | `PUT /products/1` | 200 | La réponse reflète les valeurs envoyées
Modifier partiellement | `PATCH /products/1` | 200 | La réponse reflète le champ modifié

Supprimer | `DELETE /products/1` | 200 | La réponse inclut `isDeleted: true`
Créer un utilisateur | `POST /users/add` | 201 observé | L'API renvoie un identifiant et les informations fournies
Ressource absente | `GET /products/99999` | 404 | Le serveur signale que le produit n'existe pas
JSON mal formé | `POST /products/add` | 400 | Le serveur rejette le corps JSON invalide

Le code 200 OK indique une réponse réussie. 201 Created indique une création. 400 Bad Request signale une requête invalide et 404 Not Found une ressource absente. La requête de suppression renvoie ici 200 avec un objet de confirmation, même si d'autres API utilisent 204 No Content.

Création et simulation

La requête POST de produit envoie :

```
{
"title": "Casque Audio Sans Fil",
"price": 129.99,
"category": "audio",
"stock": 25
}
```

Le script Postman vérifie le statut 200 ou 201, la présence d'un identifiant et les principales valeurs renvoyées, puis enregistre l'identifiant dans createdProductId.

Les requêtes POST, PUT, PATCH et DELETE de DummyJSON sont des simulations. Elles servent à apprendre le format des appels et des réponses, mais ne modifient pas durablement les données du service.

5. Tests automatisés

Des scripts Postman vérifient les codes de statut, la structure JSON et les valeurs importantes retournées. La requête de liste contient aussi un test de temps de réponse inférieur à 500 ms. Ce seuil dépend du réseau et de la charge du service distant ; un échec peut donc provenir de la latence plutôt que d'une erreur de configuration.

La requête de création accepte les statuts 200 et 201, car la description du TP et les API peuvent différer sur le statut exact d'une création réussie.

6. Conclusion

Ce TP m'a permis de pratiquer les méthodes HTTP courantes, les paramètres de requête, les en-têtes, les corps JSON, les codes de statut, les environnements Postman et les assertions JavaScript. L'API de démonstration est utile pour apprendre à construire des requêtes, mais une vraie application backend serait nécessaire pour vérifier une persistance permanente.

7. Captures d'écran

Les fichiers livrés contiennent la collection importable et les résultats observés par requête. Ajouter ici des captures d'écran prises dans Postman après import et exécution de la collection, si l'enseignant les exige.